Next.js Rendering Methods Demo

SSG

Static Site Generation - Pages built at build time

- Fastest performance
- Great for SEO
- Content doesn't change often

View SSG Example

SSR

Server-Side Rendering - Pages built on each request

- Always fresh data
- Good for SEO
- Dynamic content

View SSR Example

ISR

Incremental Static Regeneration - Best of both worlds

- Fast like SSG
- Updates periodically
- Scalable

View ISR Example

Key Differences

Method	When Built	Performance	Data Freshness	Use Case
SSG	Build time	Fastest	Static	Blogs, marketing pages
SSR	Each request	Slower	Always fresh	User dashboards, real-time data
ISR	Build + periodic	Fast	Periodically fresh	E-commerce, news sites

← Back to Home

SSG - Static Site Generation

> **How it works:** This page was generated at build time. The API call to JSONPlaceholder happened during the build process, not when you visited this page.

Pros: Fastest performance, great SEO, CDN cacheable

Cons: Data is static until next build

Users (Fetched at Build Time)

Leanne Graham
@Bret
Sincere@april.biz
Gwenborough

Ervin Howell
@Antonette
Shanna@melissa.tv
Wisokyburgh

Clementine Bauch
@Samantha
Nathan@yesenia.net
McKenziehaven

Patricia Lebsack
@Karianne
Julianne.OConner@kory.org
South Elvis

Chelsey Dietrich
@Kamren
Lucio_Hettinger@annie.ca
Roscoeview

Mrs. Dennis Schulist
@Leopoldo_Corkery
Karley_Dach@jasper.info
South Christy

Kurtis Weissnat
@Elwyn.Skiles
Telly.Hoeger@billy.biz
Howemouth

Nicholas Runolfsdottir V
@Maxime_Nienow
Sherwood@rosamond.me
Aliyaview

Glenna Reichert
@Delphine
Chaim_McDermott@dana.io
Bartholomebury

Clementina DuBuque
@Moriah.Stanton
Rey.Padberg@karina.biz
Lebsackbury

Code Example:

```
// This is the default behavior in Next.js App Router
export default async function SSGPage() {
  // Fetch happens at BUILD TIME
  const response = await fetch('https://jsonplaceholder.typicode.com/users
  const users = await response.json()

  return <div>{/* Render users */}</div>
}
```

← Back to Home

SSR - Server-Side Rendering

> **How it works:** This page is generated on every request. The API call to JSONPlaceholder happens each time someone visits this page. Refresh to see the timestamp update!

Pros: Always fresh data, good SEO, personalized content

Cons: Slower than SSG, server load on each request

Users (Fetched on Each Request)

Leanne Graham
@Bret
Sincere@april.biz
Gwenborough

Ervin Howell
@Antonette
Shanna@melissa.tv
Wisokyburgh

Clementine Bauch
@Samantha
Nathan@yesenia.net
McKenziehaven

Patricia Lebsack
@Karianne
Julianne.OConner@kory.org
South Elvis

Chelsey Dietrich
@Kamren
Lucio_Hettinger@annie.ca
Roscoeview

Mrs. Dennis Schulist
@Leopoldo_Corkery
Karley_Dach@jasper.info
South Christy

Kurtis Weissnat
@Elwyn.Skiles
Telly.Hoeger@billy.biz
Howemouth

Nicholas Runolfsdottir V
@Maxime_Nienow
Sherwood@rosamond.me
Aliyaview

Glenna Reichert
@Delphine
Chaim_McDermott@dana.io
Bartholomebury

Clementina DuBuque
@Moriah.Stanton
Rey.Padberg@karina.biz
Lebsackbury

Code Example:

```
export default async function SSRPage() {
  // Fetch happens on EVERY REQUEST
  const response = await fetch('https://api.example.com/users', {
    cache: 'no-store' // This forces SSR
  })
  const users = await response.json()

  return <div>{/* Render users */}</div>
}
```

← Back to Home

ISR - Incremental Static Regeneration

> **How it works:** This page was generated at build time, but it regenerates every 60 seconds when there's traffic. You get the speed of SSG with the freshness of SSR!

Revalidation: Every 60 seconds (when visited)

Pros: Fast like SSG, updates automatically, scalable

Cons: Slightly more complex, eventual consistency

Users (ISR with 60s Revalidation)

Leanne Graham
@Bret
Sincere@april.biz
Gwenborough

Ervin Howell
@Antonette
Shanna@melissa.tv
Wisokyburgh

Clementine Bauch
@Samantha
Nathan@yesenia.net
McKenziehaven

Patricia Lebsack
@Karianne
Julianne.OConner@kory.org

South Elvis

Chelsey Dietrich
@Kamren
Lucio_Hettinger@annie.ca
Roscoeview

Mrs. Dennis Schulist
@Leopoldo_Corkery
Karley_Dach@jasper.info
South Christy

Kurtis Weissnat
@Elwyn.Skiles
Telly.Hoeger@billy.biz
Howemouth

Nicholas Runolfsdottir V
@Maxime_Nienow
Sherwood@rosamond.me
Aliyaview

Glenna Reichert
@Delphine
Chaim_McDermott@dana.io
Bartholomebury

Clementina DuBuque
@Moriah.Stanton
Rey.Padberg@karina.biz
Lebsackbury

Code Example:

```
export default async function ISRPage() {
  // Fetch happens at build time, then revalidates every 60 seconds
  const response = await fetch('https://api.example.com/users', {
    next: { revalidate: 60 } // ISR with 60 second revalidation
  })
  const users = await response.json()

  return <div>{/* Render users */}</div>
}
```

ISR Behavior:

- First visitor gets the cached version (fast)
- If cache is older than 60s, regeneration starts in background
- Next visitor gets the updated version
- No visitors = no regeneration (saves resources)